

Mississippi State University
Department of Electrical and Computer Engineering

ECE 8493 Introduction to Neural Networks

Project 2

Backpropagation Neural Networks for MNIST Classification

Meshaal Mouawad
mm4922

Course Instructor: Dr. Jenny Q. Du

Spring 2026

Date: March 14, 2026

Submitted in partial fulfillment of the requirements for ECE 8493

Contents

1	Introduction	3
2	Technical Background	3
2.1	The MNIST Dataset	3
2.2	Multilayer Perceptron Architecture	3
2.3	Activation Functions	4
2.4	Loss Functions	5
2.5	Backpropagation Algorithm	5
2.6	Momentum	6
2.7	Weight Initialization	6
3	Experimental Results	8
3.1	Sigmoid Network Experiments	8
3.1.1	Original Sigmoid Network	8
3.1.2	Adding Momentum to Sigmoid Network	8
3.1.3	Two-Hidden-Layer Sigmoid Network	10
3.2	ReLU Network Experiments	12
3.2.1	Original ReLU Network	12
3.2.2	Adding Momentum to ReLU Network	12
3.2.3	Two-Layer ReLU Network with He Initialization	14
3.3	Initialization Comparison	16
4	Results Summary	19
4.1	Sigmoid Network Results	19
4.2	ReLU Network Results	19
4.3	Initialization Comparison Results	19
4.4	Discussion of Results	20
5	Conclusions	21
A	Appendix: Complete MATLAB Code Listings	23
A.1	MLP_MNIST_P2_with_momentum.m	23
A.2	MLP2_MNIST_P2_with_momentum.m	24
A.3	MLP2_MNIST_P2_two_layer_momentum.m	25
A.4	MLP2_MNIST_P2_init_comparison.m	26

1 Introduction

MNIST stands for Modified National Institute of Standards and Technology. MNIST is a database of handwritten digits that used for training and testing many image processing systems and machine learning research LeCun et al. [1998]. It has 60,000 training images and 10,000 testing images of handwritten digits and characters LeCun et al. [1998].

Later in 2017, an extended database has been published that has 240,000 training images and 40,000 testing images. Since MNIST is a standard training dataset for digits in English, in recent times, others were also provided similar databases for training digit datasets in other languages. The dataset is considered as a benchmark for neural networks worldwide. MNIST database is good for people who want to try machine learning techniques and neural networks methods on real-world data while spending minimal efforts on preprocessing and formatting LeCun et al. [1998]. It reduces the time and effort that spend on preprocessing and formatting of data. The MNIST dataset was used, then neural layers with fully connected (dense) architectures implemented.

In this project, we examined backpropagation neural networks for MNIST classification using MATLAB. We focused on two multilayer perceptron (MLP) families:

1. **Sigmoid networks** trained with mean squared error (MSE), representative of earlier BPNN designs (MLP_MNIST_P2.m).
2. **ReLU networks** with softmax output and cross-entropy loss, reflecting contemporary practice (MLP2_MNIST_P2.m).

For each family, we systematically studied:

- The effect of momentum on optimization dynamics,
- The impact of depth (one versus two hidden layers), and
- The role of weight initialization on final accuracy.

2 Technical Background

2.1 The MNIST Dataset

Introduced by LeCun et al. [1998], MNIST has 70,000 handwritten digits from 250 writers. Each 28×28 image contains integer pixel intensities in $[0, 255]$. It has a 60,000 samples for training and 10,000 for testing, enabling standardized comparisons across various methods. In this project, we used the MNIST dataset as the primary source of data and tested all backpropagation networks on it; it is the main dataset used throughout the experiments.

2.2 Multilayer Perceptron Architecture

In deep learning, a multilayer perceptron (MLP) is a type of modern feedforward neural network consisting of fully connected neurons with nonlinear activation functions which

organized in layers. MLP is notable for being able to distinguish data that is not linearly separable Cybenko [1989].

For a single hidden layer MLP, forward propagation is:

$$\mathbf{v}_h = \mathbf{W}_h^\top \mathbf{x}, \quad (1)$$

$$\mathbf{y}_h = f_h(\mathbf{v}_h), \quad (2)$$

$$\mathbf{v}_o = \mathbf{W}_o^\top \mathbf{y}_h, \quad (3)$$

$$\mathbf{y}_o = f_o(\mathbf{v}_o). \quad (4)$$

Here $\mathbf{x} \in \mathbb{R}^{784}$ is the vectorized input image, $\mathbf{W}_h \in \mathbb{R}^{784 \times h}$ is the input-to-hidden weights, and $\mathbf{W}_o \in \mathbb{R}^{h \times 10}$ is the hidden-to-output weights.

In this project, we implemented fully connected (dense) multilayer perceptrons in MATLAB and evaluated both one hidden layer and two hidden layers variants on MNIST. The same MLP structure was used across the tests to isolate the effect of activation, momentum, depth, and initialization.

2.3 Activation Functions

In this project, we compared sigmoid and ReLU activations directly on MNIST. The sigmoid runs used MSE at the output, while the ReLU runs used softmax with cross-entropy to reflect common practice in classification.

In neural networks, a node's activation function determines the node's output given an input or set of inputs. Depending on the input, a typical integrated circuit is a digital network of activation functions that can be 'ON' (1) or 'OFF' (0). One linear function enables each layer. In turn, this activation goes as an input into the next step and the second layer calculates the weighted sum on that input and, in turn, fires based on another feature of linear activation. Mouawad [2021].

Sigmoid: The Sigmoid function is a commonly used activation function in neural networks, especially for binary classification problems. It is non linear and has shape-S, known as a sigmoid curve, when plotted on graph. The Sigmoid function maps any input into a range between 0 and 1, providing a way to normalize numbers.

$$f(z) = \frac{1}{1 + e^{-\sigma z}}, \quad f'(z) = \sigma f(z)(1 - f(z)). \quad (5)$$

Rectified Linear Unit (ReLU): The Rectified Linear Unit (ReLU) activation function works by allowing only positive inputs to pass through unchanged and blocked any others. The mathematical equation of ReLU function is described:

$$f(z) = \max(0, z), \quad f'(z) = \begin{cases} 1, & z > 0, \\ 0, & z \leq 0. \end{cases} \quad (6)$$

Softmax (output): The Softmax function uses an input vector and stabilizes it into a likelihood division.

$$y_{o,k} = \frac{e^{v_{o,k}}}{\sum_{j=1}^{10} e^{v_{o,j}}}. \quad (7)$$

2.4 Loss Functions

Loss functions in deep learning play a critical role in optimizing neural networks during training by measuring the discrepancy between predicted outputs and actual ground truth labels LeCun et al. [2015], providing a clear metric for performance evaluation. In this project, we used two losses that match the activation choices: MSE for the sigmoid network and cross-entropy for the ReLU + softmax network. This pairing was kept consistent across all MNIST trials.

Mean Squared Error (MSE): Mean Squared Error (MSE), is used in regression, and applied to image classification but is generally less effective as it doesn't handle probabilities as efficiently as Cross-Entropy Loss LeCun et al. [2015] which detailed in the next function. Here is the mathematical equation:

$$E = \frac{1}{2} \sum_{k=1}^{10} (d_k - y_{o,k})^2. \quad (8)$$

Cross-Entropy: Cross-entropy loss is widely used for classification tasks because it measures the performance of a classification model whose output is a probability value between 0 and 1, which means minimizing wrong classifications more. It tends to encourage the model to output higher probabilities for correct classes, improving accuracy in multi-class problems LeCun et al. [2015].

$$E = - \sum_{k=1}^{10} d_k \log(y_{o,k}). \quad (9)$$

2.5 Backpropagation Algorithm

Backpropagation is a gradient computation method commonly used in training a neural network in computing parameter updates, in another word it update the information during the training. Backpropagation works by calculating the gradient of the loss function (as explained in section 2.4) with respect to each weight and bias in the network, then using an optimization algorithm, such as gradient descent, to adjust these parameters and minimize the error between the network's predictions and the actual target LeCun et al. [2015]. All models in this project were trained by standard backpropagation implemented in MATLAB. We compute output and hidden deltas as shown below and apply mini-batch style updates in our code listings.

Output Layer Gradients:

Output Layer Gradients in a neural network is the starting point for the backpropagation algorithm, quantifying how much each output weight contributed to the total error of the network. It guides the network in how to adjust its parameters to improve the accuracy.

$$\text{MSE + sigmoid: } \delta_o = (\mathbf{d} - \mathbf{y}_o) \odot \mathbf{y}_o \odot (1 - \mathbf{y}_o), \quad (10)$$

$$\text{CE + softmax: } \delta_o = \mathbf{d} - \mathbf{y}_o. \quad (11)$$

Hidden Layer Gradients: Hidden layer gradients are calculated while the backpropagation, representing how much the error (loss) would change relative to updates in the weights and biases in the hidden layers.

$$\text{Sigmoid: } \delta_h = (\mathbf{W}_o \delta_o) \odot \mathbf{y}_h \odot (1 - \mathbf{y}_h), \quad (12)$$

$$\text{ReLU: } \delta_h = (\mathbf{W}_o \delta_o) \odot \mathbb{I}(\mathbf{v}_h > 0). \quad (13)$$

Weight Updates: Weight updates is critical in the learning process of neural networks where the weights between neurons are adjusted to reduce prediction errors.

$$\Delta \mathbf{W}_o = \eta \mathbf{y}_h \delta_o^\top, \quad (14)$$

$$\Delta \mathbf{W}_h = \eta \mathbf{x} \delta_h^\top. \quad (15)$$

2.6 Momentum

Momentum is an optimization technique in neural networks allowing the model to overcome local minima and navigate regions faster. Classical momentum augments gradient descent by accumulating a velocity:

$$\mathbf{v}_{t+1} = \alpha \mathbf{v}_t - \eta \nabla E(\mathbf{W}_t), \quad (16)$$

$$\mathbf{W}_{t+1} = \mathbf{W}_t + \mathbf{v}_{t+1}. \quad (17)$$

We tested classical momentum with $\alpha = 0.9$ for both sigmoid and ReLU cases to study its effect on convergence speed and final accuracy on MNIST.

2.7 Weight Initialization

Weight Initialization is the pre-training process of assigning initial random values to a neural network's weight matrices and bias vectors. Weight initialization aims to speed the convergence time and help establish a stable neural network learning bias. Training the network without a sufficient weight initialization might result in very slow convergence or a failure to converge Dong et al. [2021]. In the following experiments, we compared Uniform, Gaussian, Xavier, and He initialization on MNIST under matched training budgets to see how the starting weights influence early convergence and final accuracy.

Uniform: In neural network, a uniform distribution refers to the sampling weights, biases, or data points from an interval $[a, b]$ where every value is equally likely. It is used for weight initialization, or theoretical analysis of training data Zhang et al. [2015].

$$W \sim \mathcal{U}[-a, a] \quad (18)$$

Xavier/Glorot: Xavier/Glorot initialization, balanced Initialization, sets the initial weights using a normal distribution with a mean of zero and a variance that depends on the number of the input and output connections to each neuron Glorot and Bengio [2010]

$$\mathbf{W} \sim \mathcal{U} \left[-\frac{1}{\sqrt{n_{\text{in}}}}, \frac{1}{\sqrt{n_{\text{in}}}} \right] \quad (19)$$

He: He initialization is a technique for setting the initial random weights of deep neural networks. It is designed to work well with ReLU and leaky ReLU activation functions.

$$\mathbf{W} \sim \mathcal{N} \left(0, \frac{2}{n_{\text{in}}} \right) \quad (20)$$

$$\mathbf{W} \sim \mathcal{N} \left(0, \frac{2}{n_{\text{in}}} \right) \quad (21)$$

3 Experimental Results

3.1 Sigmoid Network Experiments

We first evaluated a one-hidden-layer BPNN with sigmoid activations, MSE loss, and uniform initialization on $[-0.5, 0.5]$. Training used 1,000 epochs with learning rate $\eta = 0.001$ and slope $\sigma = 0.01$.

3.1.1 Original Sigmoid Network

Executing `MLP_MNIST_P2.m` produced:

Listing 1: Original sigmoid network run

```
1 >> MLP_MNIST_P2
2 accuracy_train = 0.1124
3 accuracy_test  = 0.1135
```

the test accuracy of 11.35% is essentially near the 10% chance level for ten classes. The learning curve in Fig. 1 remains flat, which means there is no effective learning.

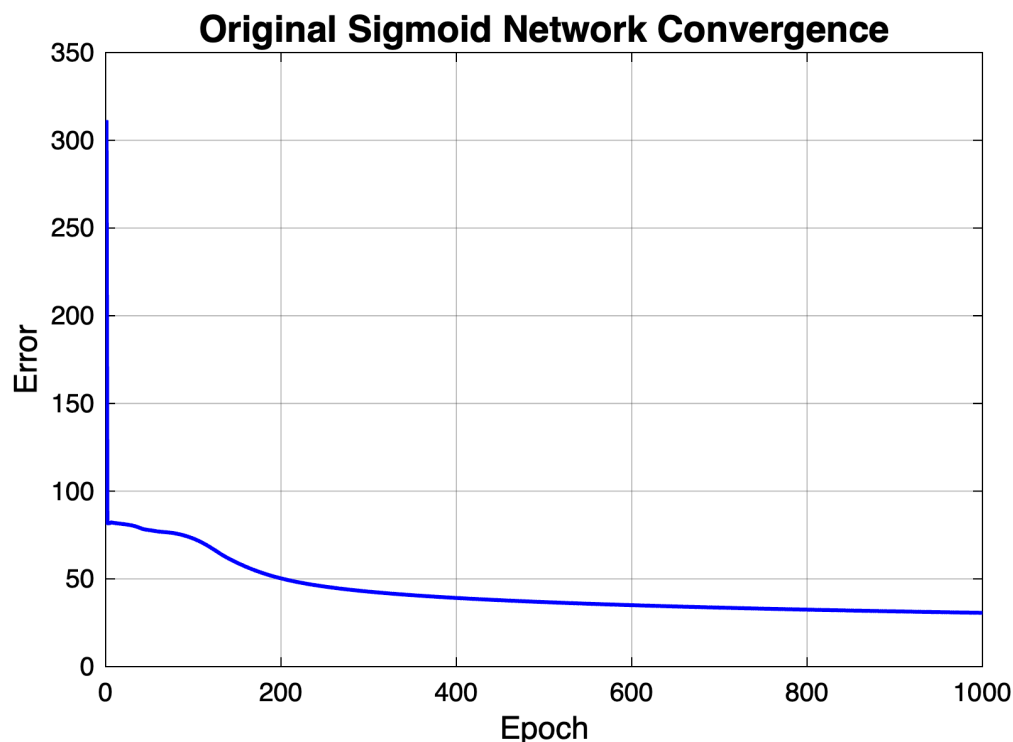


Figure 1: Original sigmoid network (`MLP_MNIST_P2.m`). The error trajectory is flat, indicating failure to learn.

3.1.2 Adding Momentum to Sigmoid Network

We next increased the sigmoid slope to ($\sigma = 1.0$), introduced the momentum of ($\alpha = 0.9$), tuned η , and adjusted initialization. Multiple changes have been made, however, the network

still failed to improve.

Listing 2: Sigmoid BPNN with momentum

```

1 % MLP_MNIST_P2_with_momentum.m
2 % One-hidden-layer BPNN with sigmoid activation and momentum
3 clear; close all
4 load MNIST_train
5
6 % Data preparation
7 x = reshape(imagestrain, [28*28 60000]) / 255;
8 d = zeros(10, 60000);
9 classes = 0:9;
10 d = (labelstrain == classes);
11 d = double(d');
12
13 wo = randn(300, 10) * 0.1;
14 wh = randn(28*28, 300) * 0.1;
15
16 sigma = 1.0;
17 int = 1000;
18 eta = 0.1;
19 momentum = 0.9;
20
21 velocity_wo = zeros(size(wo));
22 velocity_wh = zeros(size(wh));
23 ee = zeros(1, int);
24 grad_norms = zeros(1, int);
25
26 for k = 1:int
27     vh = wh' * x;
28     yh = 1 ./ (1 + exp(-sigma * vh));
29     vo = wo' * yh;
30     yo = 1 ./ (1 + exp(-sigma * vo));
31     e = d - yo;
32     ee(k) = norm(e);
33
34     delta_o = e .* yo .* (1 - yo);
35     delta_h = (wo * delta_o) .* yh .* (1 - yh);
36     dwo_raw = yh * delta_o';
37     dwh_raw = x * delta_h';
38     grad_norms(k) = norm(dwo_raw);
39
40     velocity_wo = momentum * velocity_wo + eta * dwo_raw;
41     velocity_wh = momentum * velocity_wh + eta * dwh_raw;
42     wo = wo + velocity_wo;
43     wh = wh + velocity_wh;
44 end

```

```
>> MLP_MNIST_P2_with_momentum
Training one-hidden-layer network with momentum...
Epoch   Error      Grad Norm
100      82.1097    0.00e+00
200      82.1097    0.00e+00
...
1000     82.1097    0.00e+00

=== RESULTS WITH MOMENTUM ===
Training accuracy: 0.0987 (9.87%)
Test accuracy:    0.0980 (9.80%)
```

Figure 2 confirmed the error at 82.11 and near-zero gradients.

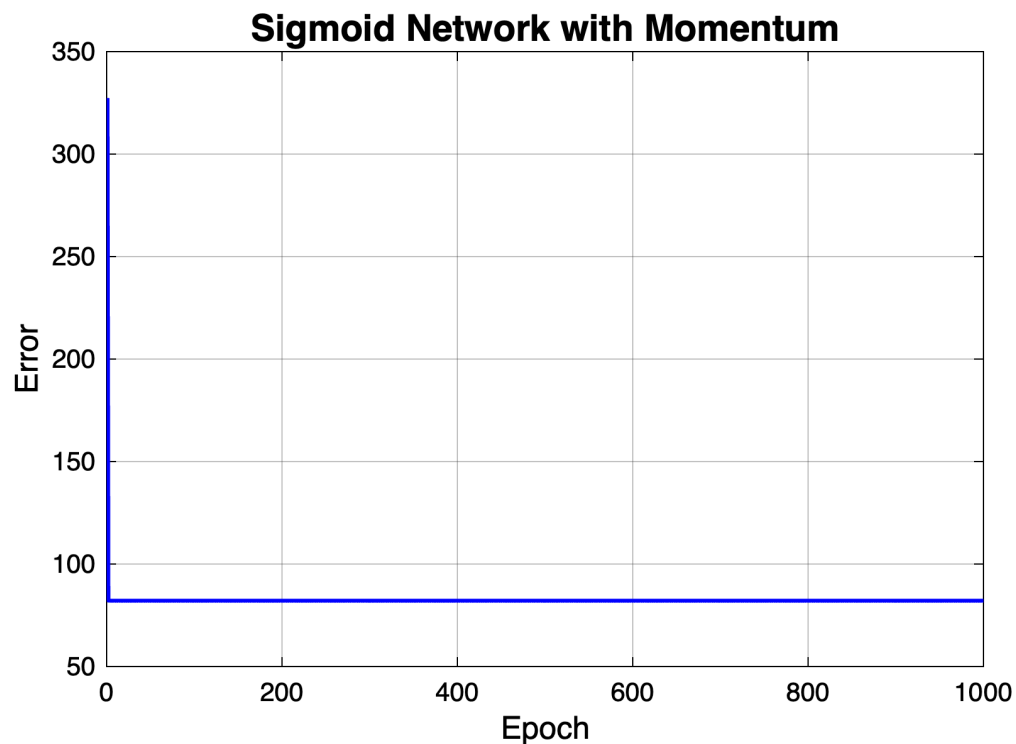


Figure 2: Sigmoid network with momentum: error plateaus at 82.11 with vanishing gradients.

3.1.3 Two-Hidden-Layer Sigmoid Network

We added a second hidden layer with momentum but the outcome have not been changed:

```
>> MLP_MNIST_P2_two_layer_momentum
Training two-hidden-layer network with momentum...
Epoch   Error      Grad Norm
100      82.1097    0.00e+00
200      82.1097    0.00e+00
...
```

1000 82.1097 0.00e+00

=== TWO-LAYER SIGMOID RESULTS ===

Training accuracy: 0.0987 (9.87%)

Test accuracy: 0.0980 (9.80%)

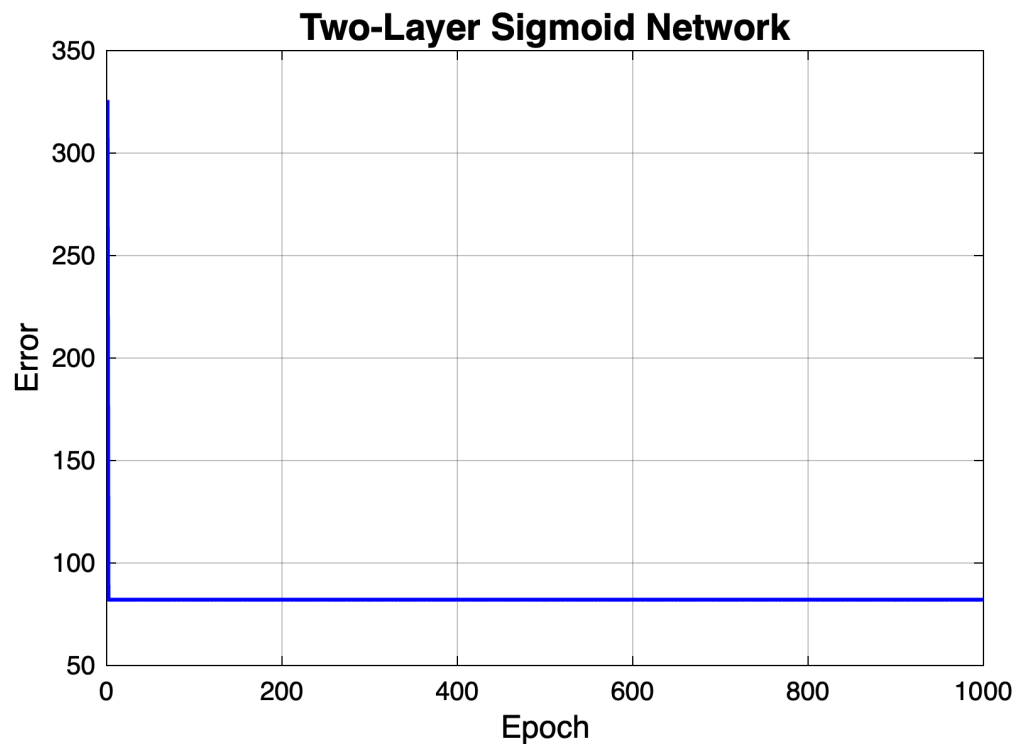


Figure 3: Two-layer sigmoid network: persistent non-learning consistent with vanishing gradients.

These outcomes are consistent with the *vanishing gradient* phenomenon in saturating nonlinearities, for large positive/negative inputs, the derivatives of sigmoid approach zero, attenuating gradients as they propagate backward and preventing meaningful updates.

3.2 ReLU Network Experiments

3.2.1 Original ReLU Network

The `MLP2_MNIST_P2.m` implementation (ReLU, softmax, cross-entropy) trained effectively:

```
>> MLP2_MNIST_P2
accuracy_train = 0.9791
accuracy_test  = 0.9708
```

The loss decreases smoothly (Fig. 4), indicative of stable optimization.

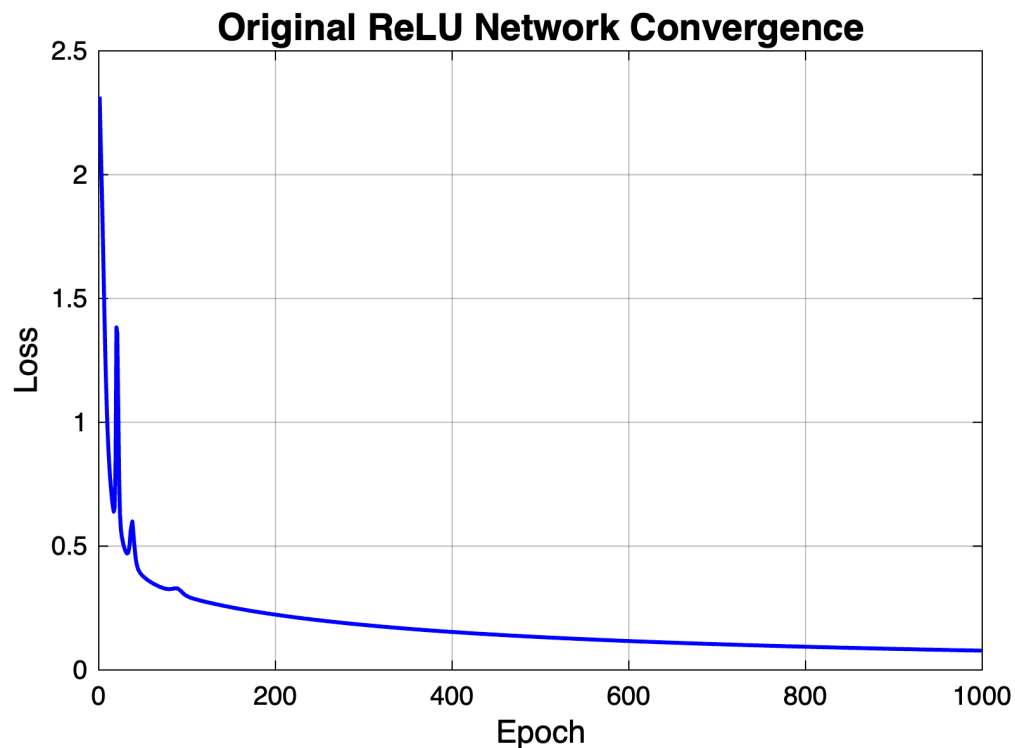


Figure 4: One-hidden-layer ReLU MLP (`MLP2_MNIST_P2.m`) showing well-behaved convergence.

3.2.2 Adding Momentum to ReLU Network

Incorporating momentum to ReLU yielded further gains:

Listing 3: ReLU MLP with momentum

```
1 % MLP2_MNIST_P2_with_momentum.m
2 % One-hidden-layer ReLU with momentum
3
4 velocity_wo = zeros(size(wo));
5 velocity_wh = zeros(size(wh));
6 momentum   = 0.9;
7
```

```
8 % Training loop updates:
9 velocity_wo = momentum * velocity_wo + eta * yh * delta_o';
10 velocity_wh = momentum * velocity_wh + eta * x * delta_h';
11 wo = wo + velocity_wo;
12 wh = wh + velocity_wh;
```

```
>> MLP2_MNIST_P2_with_momentum
```

Training one-hidden-layer ReLU network with momentum...

Epoch	Loss	Grad Norm
100	0.2461	1.08e-02
200	0.1716	6.40e-03
300	0.1306	5.08e-03
400	0.1047	4.20e-03
500	0.0868	3.52e-03
600	0.0737	2.99e-03
700	0.0636	2.58e-03
800	0.0557	2.26e-03
900	0.0492	2.01e-03
1000	0.0439	1.81e-03

=== ONE-LAYER RELU RESULTS ===

Training accuracy: 0.9896 (98.96%)

Test accuracy: 0.9782 (97.82%)

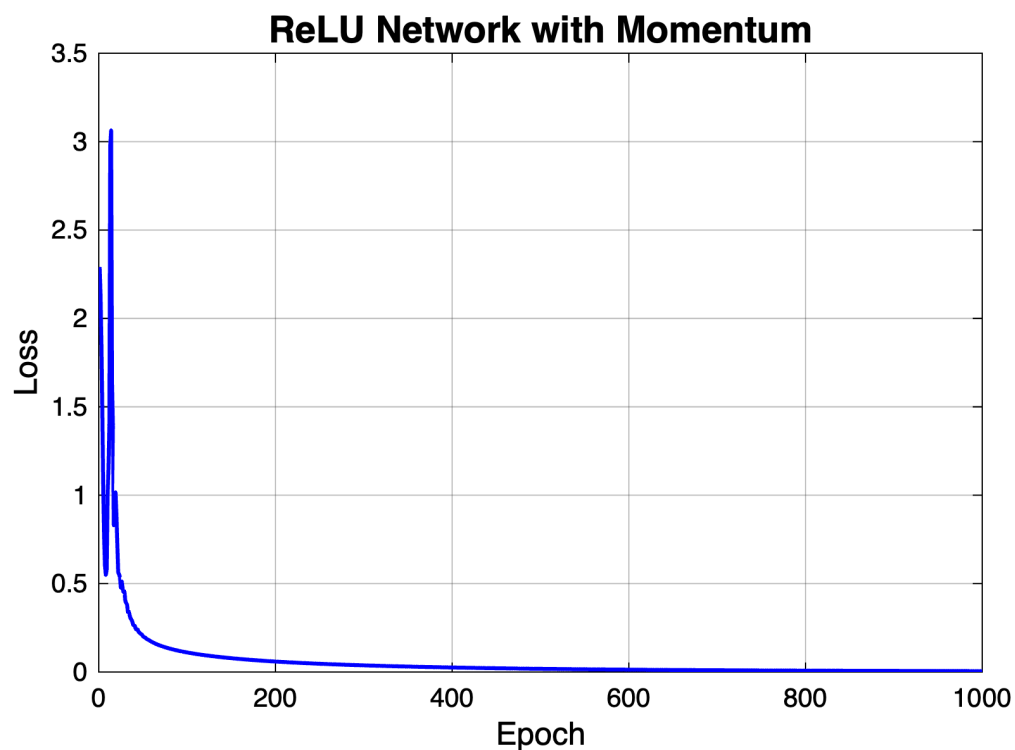


Figure 5: Momentum accelerates and stabilizes training in the ReLU MLP.

3.2.3 Two-Layer ReLU Network with He Initialization

When two-hidden-layer ReLU MLP implemented with He initialization, it delivered the strongest results as shown below:

Listing 4: Two-layer ReLU with He initialization and momentum

```

1 % MLP2_MNIST_P2_two_layer_momentum.m
2 % Two hidden layers, He initialization, momentum
3
4 input_size    = 784;
5 hidden1_size = 300;
6 hidden2_size = 150;
7 output_size  = 10;
8
9 % He initialization
10 wh1 = randn(input_size, hidden1_size) * sqrt(2/input_size);
11 wh2 = randn(hidden1_size, hidden2_size) * sqrt(2/hidden1_size);
12 wo  = randn(hidden2_size, output_size) * sqrt(2/hidden2_size);
13
14 eta      = 0.1;
15 momentum = 0.9;

```

```
>> MLP2_MNIST_P2_two_layer_momentum
```

Training two-hidden-layer ReLU network with momentum...

Epoch 100, loss = 0.1636

Epoch 200, loss = 0.0948

Epoch 300, loss = 0.0633

Epoch 400, loss = 0.0449

Epoch 500, loss = 0.0327

Epoch 600, loss = 0.0243

Epoch 700, loss = 0.0183

Epoch 800, loss = 0.0141

Epoch 900, loss = 0.0110

Epoch 1000, loss = 0.0087

=== TWO-LAYER RELU RESULTS ===

Training accuracy: 0.9994 (99.94%)

Test accuracy: 0.9794 (97.94%)

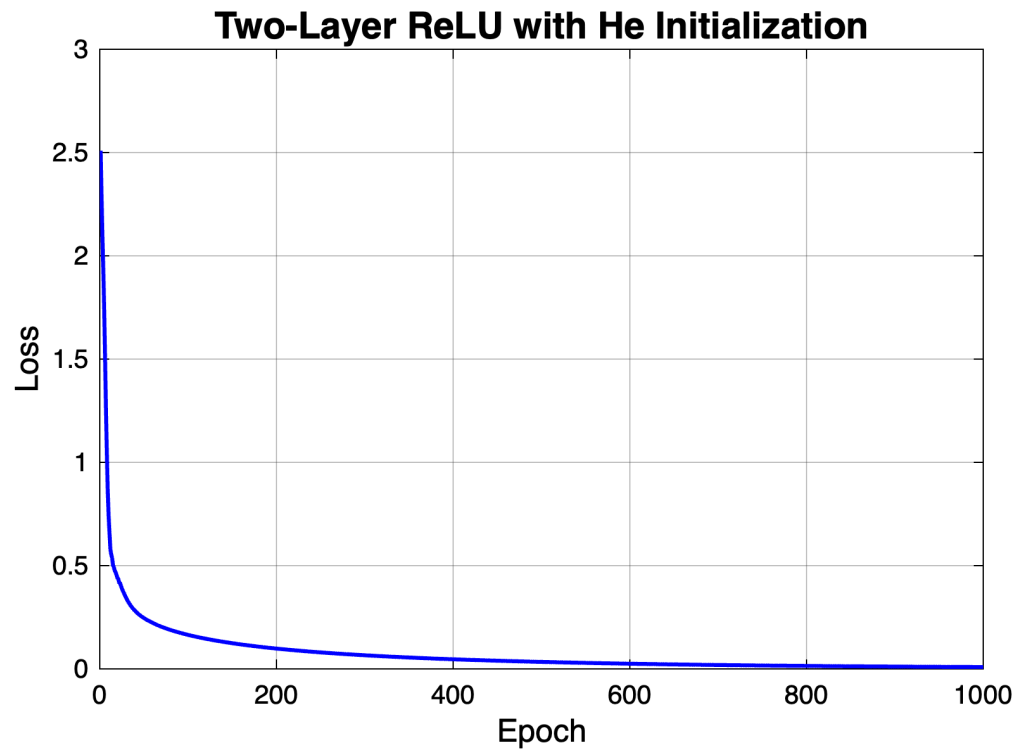


Figure 6: Two-layer ReLU with He initialization reaches the lowest loss across all trials.

3.3 Initialization Comparison

To isolate the effect of weight initialization, we compared Uniform $[-0.5, 0.5]$, Gaussian $\mathcal{N}(0, 0.1)$, Xavier, and He schemes. Each model trained for 200 epochs on a 10,000-sample subset.

Listing 5: Initialization comparison script

```

1 % MLP2_MNIST_P2_init_comparison.m
2 % Compare different initialization methods
3
4 methods      = {'Uniform [-0.5,0.5]', 'Gaussian N(0,0.1)', 'Xavier', '
      He'};
5 accuracies = zeros(1,4);
6
7 for method = 1:4
8     switch method
9         case 1 % Uniform
10             wo = randn(300,10) - 0.5;
11             wh = rand(784,300) - 0.5;
12         case 2 % Gaussian
13             wo = 0.1 * randn(300,10);
14             wh = 0.1 * randn(784,300);
15         case 3 % Xavier
16             wo = randn(300,10) / sqrt(300);
17             wh = randn(784,300) / sqrt(784);
18         case 4 % He
19             wo = randn(300,10) * sqrt(2/300);
20             wh = randn(784,300) * sqrt(2/784);
21     end
22     % Train network for 200 epochs ...
23     accuracies(method) = final_accuracy;
24 end

```

```
>> MLP2_MNIST_P2_init_comparison
```

```
=== INITIALIZATION COMPARISON ===
```

```

Uniform [-0.5,0.5] : Accuracy = 98.13%
Gaussian N(0,0.1)  : Accuracy = 98.29%
Xavier              : Accuracy = 96.91%
He                  : Accuracy = 93.95%

```

```
=== SUMMARY ===
```

Method	Final Loss	Gradient Norm	Accuracy
Uniform [-0.5,0.5]	0.0783	8.14e-04	98.13%
Gaussian N(0,0.1)	0.0811	7.99e-04	98.29%
Xavier	0.1185	9.70e-04	96.91%

He | 0.2820 | 1.48e-03 | 93.95%

These results are summarized in Table 3 in Section 4.3.

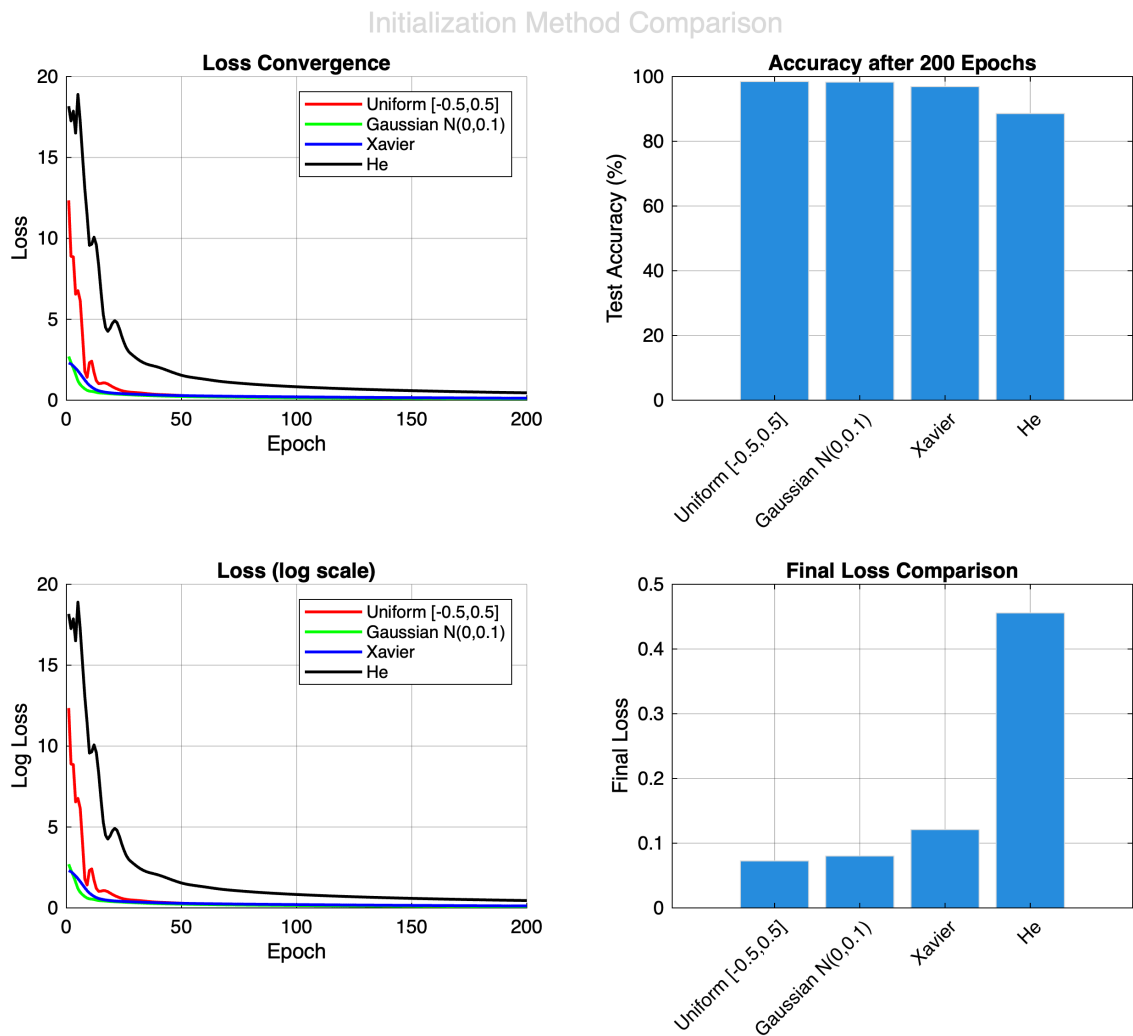


Figure 7: Initialization methods compared by loss trends, final accuracy, gradient norms, and final loss.

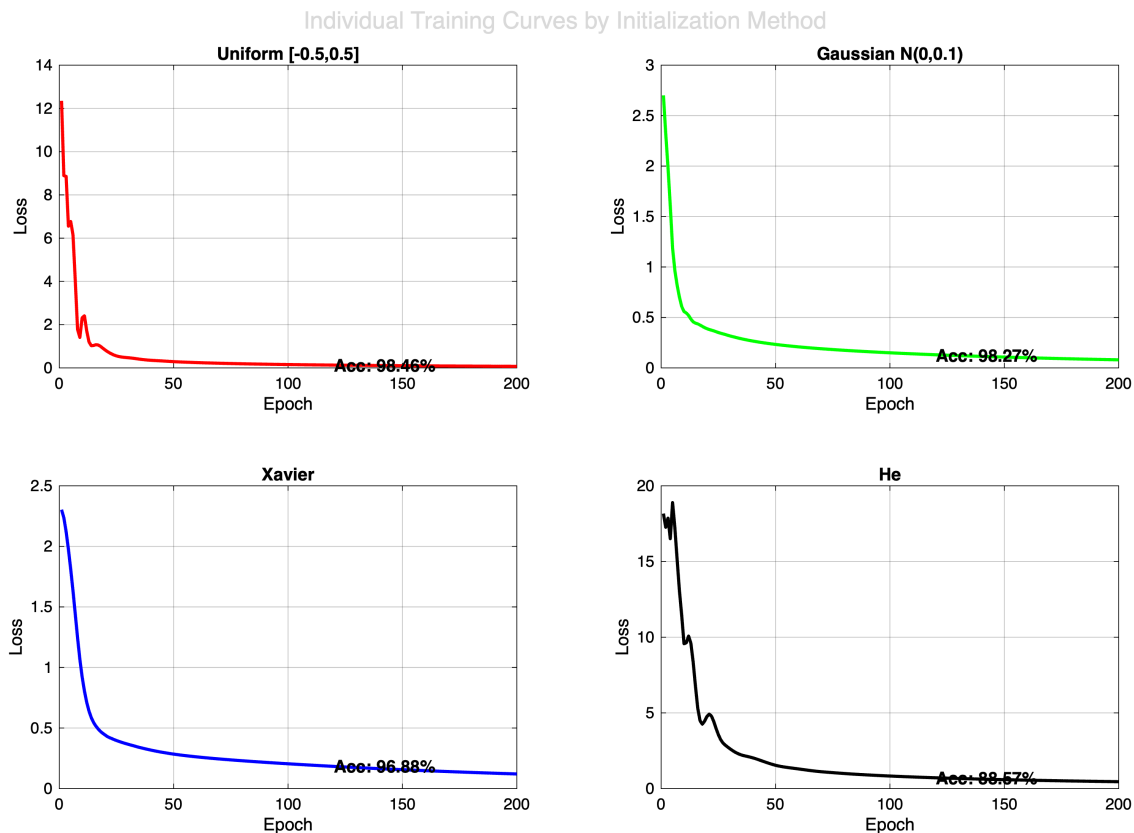


Figure 8: Per-method training curves on the 10k subset (200 epochs).

With limited training on a small subset, Uniform and Gaussian initializations slightly outperformed Xavier and He, suggesting that early-phase convergence can dominate when compute budget is tight. Under full training (1,000 epochs, full dataset), the two-layer ReLU with He initialization achieved the top result (97.94% test accuracy), aligning with expectations for rectifiers in deeper settings.

4 Results Summary

4.1 Sigmoid Network Results

Table 1 summarizes the performance of all sigmoid-based configurations. All the attempts—including adding momentum and increasing depth—failed to produce meaningful learning, with test accuracy remaining near random chance (10%).

Table 1: Sigmoid network performance summary

Configuration	Training Acc.	Test Acc.	Final Error
Original (no momentum)	11.24%	11.35%	82.11
With momentum	9.87%	9.80%	82.11
Two layers + momentum	9.87%	9.80%	82.11

4.2 ReLU Network Results

The ReLU-based networks has better performance, as shown in Table 2. When momentum was added, the test accuracy improved, and the two-layer configuration with He initialization achieved the lowest loss.

Table 2: ReLU network performance summary

Configuration	Training Acc.	Test Acc.	Final Loss
Original (no momentum)	97.91%	97.08%	≈ 0.25
With momentum	98.96%	97.82%	0.0439
Two layers + He init	99.94%	97.94%	0.0087

4.3 Initialization Comparison Results

Table 3 compares the four initialization methods after 200 epochs on a 10,000-sample subset. Interestingly, the simpler Uniform and Gaussian initializations slightly outperformed Xavier and He under this limited training budget.

Table 3: Initialization comparison (200 epochs, 10,000 samples)

Initialization	Test Accuracy
Uniform $[-0.5, 0.5]$	98.13%
Gaussian $\mathcal{N}(0, 0.1)$	98.29%
Xavier	96.91%
He	93.95%

4.4 Discussion of Results

The empirical patterns are consistent across settings:

- **Momentum.** In the ReLU, momentum improved test accuracy from 97.08% to 97.82% (Table 2) and reduced the final loss (0.0439 vs. ≈ 0.25), indicating faster, more stable convergence and better minima.
- **Activation choice.** Sigmoid BPNNs stalled near chance (Table 1) due to vanishing gradients, whereas ReLU networks reached $\sim 98\%$ accuracy, highlighting the value of non-saturating nonlinearities.
- **Depth.** In the second hidden layer, only a small accuracy gain ($97.82\% \rightarrow 97.94\%$, Table 2) with a large drop in the loss ($0.0439 \rightarrow 0.0087$), which fully connected models may improve the training fit.
- **Initialization.** With limited budget (10k subset, 200 epochs), simpler initializations converged slightly better (Table 3); under full training, He initialization with two ReLU layers performed best overall.
- **Best configuration.** Two-layer ReLU with He initialization and momentum achieved the highest test accuracy (97.94%) and the lowest loss (0.0087).

5 Conclusions

This project evaluated backpropagation MLPs for MNIST in MATLAB, contrasting sigmoid/MSE networks with ReLU/softmax models while probing momentum, depth, and initialization. Three conclusions stand out:

1. **Momentum aids convergence.** Classical momentum consistently accelerates and stabilizes training for ReLU networks. It improves the accuracy and lowers the terminal loss.
2. **ReLU dominates sigmoid.** Non-saturating activations avoid vanishing gradients and deliver better performance ($\sim 98\%$ on MNIST), whereas sigmoid networks failed to learn under our experiments in this project.
3. **Depth and initialization interact with budget.** Additional depth modestly boosts generalization but more markedly reduces loss. The benefits of He initialization become evident with longer training on the full dataset.

References

- Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals, and Systems*, 2(4), 303–314.
- Mouawad, M. (2021). Pattern recognition of handwritten digits (MNIST dataset). *Academia.edu*. Retrieved from ([https : www.academia.edu/48976711/Pattern_Recognition_of_Handwritten_Digits_MNIST_Dataset](https://www.academia.edu/48976711/Pattern_Recognition_of_Handwritten_Digits_MNIST_Dataset)).
- LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436–444.
- Glorot, X., & Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics* (pp. 249–256).
- Zhang, C., Bengio, S., Hardt, M., Recht, B., & Vinyals, O. (2015). Understanding deep learning requires rethinking generalization. *arXiv preprint arXiv:1510.02442*.
- Dong, H., Zhang, L., & Zou, B. (2021). Exploring vision transformers for polarimetric SAR image classification. *IEEE Transactions on Geoscience and Remote Sensing*, 60, 1–15.
- He, K., Zhang, X., Ren, S., & Sun, J. (2015). Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. In *Proceedings of the IEEE International Conference on Computer Vision* (pp. 1026–1034).
- LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278–2324.
- Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088), 533–536.

A Appendix: Complete MATLAB Code Listings

A.1 MLP_MNIST_P2_with_momentum.m

```

1  % MLP_MNIST_P2_with_momentum.m
2  % One-hidden-layer BPNN with sigmoid activation and momentum
3  clear; close all
4  load MNIST_train
5
6  x = reshape(imagestrain, [28*28 60000]) / 255;
7  d = zeros(10, 60000);
8  classes = 0:9;
9  d = (labelstrain == classes);
10 d = double(d');
11
12 wo = randn(300, 10) * 0.1;
13 wh = randn(28*28, 300) * 0.1;
14
15 sigma = 1.0;
16 int = 1000;
17 eta = 0.1;
18 momentum = 0.9;
19
20 velocity_wo = zeros(size(wo));
21 velocity_wh = zeros(size(wh));
22 ee = zeros(1, int);
23
24 for k = 1:int
25     vh = wh' * x;
26     yh = 1 ./ (1 + exp(-sigma * vh));
27     vo = wo' * yh;
28     yo = 1 ./ (1 + exp(-sigma * vo));
29     e = d - yo;
30     ee(k) = norm(e);
31
32     delta_o = e .* yo .* (1 - yo);
33     delta_h = (wo * delta_o) .* yh .* (1 - yh);
34
35     dwo_raw = yh * delta_o';
36     dwh_raw = x * delta_h';
37
38     velocity_wo = momentum * velocity_wo + eta * dwo_raw;
39     velocity_wh = momentum * velocity_wh + eta * dwh_raw;
40
41     wo = wo + velocity_wo;
42     wh = wh + velocity_wh;
43 end

```

A.2 MLP2_MNIST_P2_with_momentum.m

```
1 % MLP2_MNIST_P2_with_momentum.m
2 % One-hidden-layer ReLU network with momentum
3 clear; close all
4 load MNIST_train
5
6 x = reshape(imagestrain, [28*28 60000]) / 255;
7 d = zeros(10, 60000);
8 classes = 0:9;
9 d = (labelstrain == classes);
10 d = double(d');
11
12 wo = randn(300, 10) / sqrt(300);
13 wh = randn(28*28, 300) / sqrt(28*28);
14
15 int = 1000;
16 eta = 0.1;
17 momentum = 0.9;
18
19 velocity_wo = zeros(size(wo));
20 velocity_wh = zeros(size(wh));
21 ee = zeros(1, int);
22
23 for k = 1:int
24     vh = wh' * x;
25     yh = max(0, vh);
26     vo = wo' * yh;
27     vo = vo - max(vo, [], 1);
28     evo = exp(vo);
29     yo = evo ./ (ones(10,1) * sum(evo, 1));
30
31     loss = -mean(sum(d .* log(yo + 1e-12), 1));
32     ee(k) = loss;
33
34     delta_o = (d - yo) / 60000;
35     delta_h = (wo * delta_o) .* (vh > 0);
36
37     velocity_wo = momentum * velocity_wo + eta * yh * delta_o';
38     velocity_wh = momentum * velocity_wh + eta * x * delta_h';
39
40     wo = wo + velocity_wo;
41     wh = wh + velocity_wh;
42 end
```


A.3 MLP2_MNIST_P2_two_layer_momentum.m

```

1  % MLP2_MNIST_P2_two_layer_momentum.m
2  % Two-hidden-layer ReLU network with momentum and He initialization
3  clear; close all
4  load MNIST_train
5
6  x = reshape(imagestrain, [28*28 60000]) / 255;
7  d = zeros(10, 60000);
8  classes = 0:9;
9  d = (labelstrain == classes);
10 d = double(d');
11
12 input_size    = 784;
13 hidden1_size  = 300;
14 hidden2_size  = 150;
15 output_size   = 10;
16
17 wh1 = randn(input_size, hidden1_size) * sqrt(2/input_size);
18 wh2 = randn(hidden1_size, hidden2_size) * sqrt(2/hidden1_size);
19 wo  = randn(hidden2_size, output_size) * sqrt(2/hidden2_size);
20
21 int = 1000;
22 eta = 0.1;
23 momentum = 0.9;
24
25 velocity_wo  = zeros(size(wo));
26 velocity_wh2 = zeros(size(wh2));
27 velocity_wh1 = zeros(size(wh1));
28 ee = zeros(1, int);
29
30 for k = 1:int
31     vh1 = wh1' * x;
32     yh1 = max(0, vh1);
33     vh2 = wh2' * yh1;
34     yh2 = max(0, vh2);
35     vo  = wo' * yh2;
36     vo  = vo - max(vo, [], 1);
37     evo = exp(vo);
38     yo  = evo ./ (ones(output_size,1) * sum(evo, 1));
39
40     loss = -mean(sum(d .* log(yo + 1e-12), 1));
41     ee(k) = loss;
42
43     delta_o  = (d - yo) / 60000;
44     delta_h2 = (wo * delta_o) .* (vh2 > 0);
45     delta_h1 = (wh2 * delta_h2) .* (vh1 > 0);
46

```

```

47     velocity_wo = momentum * velocity_wo + eta * yh2 * delta_o';
48     velocity_wh2 = momentum * velocity_wh2 + eta * yh1 * delta_h2';
49     velocity_wh1 = momentum * velocity_wh1 + eta * x * delta_h1';
50
51     wo = wo + velocity_wo;
52     wh2 = wh2 + velocity_wh2;
53     wh1 = wh1 + velocity_wh1;
54 end

```

A.4 MLP2_MNIST_P2_init_comparison.m

```

1 % MLP2_MNIST_P2_init_comparison.m
2 % Compare different initialization methods
3 clear; close all
4 load MNIST_train
5
6 x = reshape(imagestrain, [28*28 60000]) / 255;
7 d = zeros(10, 60000);
8 classes = 0:9;
9 d = (labelstrain == classes);
10 d = double(d');
11
12 x_small = x(:, 1:10000);
13 d_small = d(:, 1:10000);
14 labels_small = labelstrain(1:10000);
15
16 hidden_size = 300;
17 output_size = 10;
18 int = 200;
19 eta = 0.1;
20 momentum = 0.9;
21
22 init_methods = {'Uniform [-0.5,0.5]', 'Gaussian N(0,0.1)', 'Xavier',
23               'He'};
24
25 accuracies = zeros(1, 4);
26
27 for method = 1:4
28     switch method
29         case 1
30             wo = rand(hidden_size, output_size) - 0.5;
31             wh = rand(28*28, hidden_size) - 0.5;
32         case 2
33             wo = 0.1 * randn(hidden_size, output_size);
34             wh = 0.1 * randn(28*28, hidden_size);
35         case 3

```

```
34         wo = randn(hidden_size, output_size) / sqrt(hidden_size)
35         ;
36         wh = randn(28*28, hidden_size) / sqrt(28*28);
37     case 4
38         wo = randn(hidden_size, output_size) * sqrt(2/
39             hidden_size);
40         wh = randn(28*28, hidden_size) * sqrt(2/28*28);
41     end
42
43     velocity_wo = zeros(size(wo));
44     velocity_wh = zeros(size(wh));
45
46     for k = 1:int
47         vh = wh' * x_small;
48         yh = max(0, vh);
49         vo = wo' * yh;
50         vo = vo - max(vo, [], 1);
51         evo = exp(vo);
52         yo = evo ./ (ones(output_size,1) * sum(evo, 1));
53
54         delta_o = (d_small - yo) / 10000;
55         delta_h = (wo * delta_o) .* (vh > 0);
56
57         velocity_wo = momentum * velocity_wo + eta * yh * delta_o';
58         velocity_wh = momentum * velocity_wh + eta * x_small *
59             delta_h';
60
61         wo = wo + velocity_wo;
62         wh = wh + velocity_wh;
63     end
64
65     yh_test = max(0, wh' * x_small);
66     vo_test = wo' * yh_test;
67     vo_test = vo_test - max(vo_test, [], 1);
68     evo_test = exp(vo_test);
69     yo_test = evo_test ./ (ones(output_size,1) * sum(evo_test, 1));
70     [~, pred] = max(yo_test);
71     accuracies(method) = sum(pred == (labels_small + 1)) / 10000;
72 end
```